

بسمه تعالی

گزارش کار هوش مصنوعی

Numerical Puzzle

استاد
آقای دکتر قویدل

نویسندگان
جواد باجلان

دانشگاه شهید مدنی آذربایجان
زمستان 1403

هدف

یافتن ماتریسی از روی ماتریس $n \times m$ ورودی با صفر نمودن بعضی از اعداد آن به طوری که جمع هر سطر و ستون برابر با جمع هدف باشد که در ورودی گرفته می‌شود.

چالش‌ها

- I. پیاده‌سازی جستجوی عقب‌گرد
- II. بررسی رو به جلو
- III. کم‌ترین مقدار باقی‌مانده (MRW)
- IV. هیوریستیک درجه
- V. پشتیبانی از ماتریس‌های غیرمربعی
- VI. نمایش فرآیند حل به صورت گام‌به‌گام
- VII. گسترش به محدودیت یکتا بودن اعداد در هر ردیف یا ستون

شرح

این مسئله جزو مسائل CSP در هوش مصنوعی قرار دارد و برای حل آن از الگوریتم‌های موجود در این دسته از مسائل استفاده خواهیم نمود.

متغیرها: هر عدد در داخل ماتریس، متغیر است. ($n \times m$ متغیر). برای پیاده‌سازی ایندکس آن را لحاظ می‌کنیم.

دامنه: دامنه هر متغیر به صورت 0 و 1 تعریف می‌شود. 0 به معنای این که باید حذف شود (صفر شود) و 1 به معنی نگه‌داری آن می‌باشد.

آزمون هدف: آیا جمع سطرها و ستون‌های ماتریس برابر جمع هدف می‌شود یا خیر.

برای مثال ماتریس زیر را در نظر بگیرید:

$$M: \begin{bmatrix} 6 & 3 \\ 4 & 8 \end{bmatrix}$$

$$\text{Row_sum} = \{6, 8\}$$

$$\text{Col_sum} = \{6, 8\}$$

متغیرها: $\{(0, 0), (0, 1), (1, 0), (1, 1)\}$

دامنه: $\{1, 0\}$

محدودیت ضمنی:

$$\sum_{i=0}^n (\sum_{j=0}^m M_{ij} = \text{Row_sum}[i])$$

$$\sum_{j=0}^m (\sum_{i=0}^n M_{ij} = \text{Col_sum}[j])$$

Uniqueness of Rows and cols

الگوریتم به این شکل است که به ازای هر متغیر، یک عضو از دامنه را به آن انتساب می‌کند و اگر در جایی مشکلی در نقض محدودیت آمد، به عقب باز می‌گردد و از عضو دیگر دامنه استفاده می‌کند.

این الگوریتم پایه برای ماتریس‌های کوچک همانند 3×3 و 4×4 مناسب است اما برای ماتریس‌های دیگر خیلی کند عمل می‌کند. بنابراین باید با ایده‌هایی آن را بهبود ببخشیم.

ایده اول: بررسی رو به جلو (forward checking).

با بررسی به ازای هر مقدار از دامنه، اگر متغیری با دامنه تهی وجود داشت، به معنی این است که در ادامه به مشکل خواهیم خورد، بنابراین از ادامه جلوگیری خواهیم نمود و مقدار دیگر از دامنه را بررسی خواهیم نمود و یا این که به عقب باز خواهیم گشت.

از آن جا که صفر کردن و یا نگه‌داری متغیر فقط در دامنه متغیرهای هم سطر و ستون تغییر ایجاد خواهد کرد، از این فکت استفاده خواهیم نمود که جمع متغیرهای مقداردهی شده نباید از جمع هدف بیشتر شود، زیرا در این صورت قطعاً جوابی برای مسئله نخواهند بود، بنابراین این شرط را برای تمام متغیرها در آن سطر و ستون بررسی می‌کنیم و هم‌چنین آیا با متغیرهای مقداردهی نشده باقی‌مانده می‌توان به جمع هدف رسید یا خیر، به صورت دیگر، آیا جمع متغیرهای باقی‌مانده از باقی‌مانده جمع هدف کم‌تر است یا خیر.

هم‌چنین برای بررسی یکتا بودن یا نبودن آن را اعمال می‌کنیم تا در صورت تکراری بودن عددی در سطر یا ستونی از ماتریس، از ادامه جلوگیری کند.

ایده دوم: هیوریستیک درجه (Heuristic Degree).

ابتدا متغیری انتخاب می‌شود که بیشترین درجه را خواهد داشت. درجه در این جا می‌توانیم تعداد متغیرهای مقداردهی نشده در سطر و ستون متغیر در نظر بگیریم.

تابع جانشین:

وظیفه انتخاب متغیر برای مقداردهی را دارد. این انتخاب را به صورتی انجام می‌دهد که

۱. دامنه آن از همه کم‌تر باشد (در این مسئله دامنه فقط دارای یک مقدار باشد).
۲. با مقداردهی، جمع متغیرهای مقداردهی شده در آن سطر از جمع هدف بیشتر نشود.
۳. با مقداردهی، جمع متغیرهای مقداردهی شده در آن ستون از جمع هدف بیشتر نشود.
۴. با مقداردهی، بتوان جمع هدف باقی‌مانده را با متغیرهای باقی‌مانده در آن سطر و ستون به دست آورد.
۵. در آن سطر و ستون وجود نداشته باشد (یکتا بودن).

usage: puzzle.py [-h] [-f F] [-o O] [-v] [-u]

number puzzle with csp

options:

- h, --help show this help message and exit*
- f F filepath to the puzzle, default: puzzle.txt*
- o O filepath to save result, default: solution.txt*
- v verbose mode*
- u unique solution*